

PARALLEL & DISTRIBUTED COMPUTING

Laboratory Examination — Windows Systems Programming

Build SysForge

A Real-Time Parallel Windows System Utility

EXAM INFORMATION

Duration: 2 Hours. The application will not be complete. That is intentional.

Total: 100 marks (+10 bonus for GPU monitoring)

Platform: Windows only. Win32 API, WMI, or PDH for hardware data.

Language: C++ (recommended) or Python with pywin32 / psutil. State your choice clearly.

Submit: ZIP of all source files + one screen recording (minimum 60 seconds) showing the app running while the examiner's stress test is active.

Open book, open internet. No AI tools. No communication with other students.

THE ONE THING THAT DETERMINES YOUR GRADE

The examiner will open Windows Task Manager alongside your application.

Then the examiner will run a CPU stress test.

Then the examiner will open a video file using your app's Process Controller.

Then the examiner will ask your app's Cleaner to scan the machine.

If your bars move when Task Manager moves: you built it correctly.

If your process list matches Task Manager's list: you built it correctly.

If your cleaner finds files that actually exist on the machine: you built it correctly.

There is no other way to pass this exam. The data must be real.

What You Are Building

SysForge is a Windows desktop system utility — a single application window with three tabs that run simultaneously. Each tab communicates with the operating system in real

time using Windows APIs. All three tabs must be live at the same time: closing or switching tabs must not stop background monitoring.

The application has one strict technical requirement above all others: the GUI thread must never be blocked. Every hardware query, every file scan, every process operation must happen in a background thread. If the examiner clicks any button and the window freezes or becomes unresponsive for more than half a second, marks are deducted from that section immediately.

The three tabs at a glance

Tab 1 — Monitor: Live CPU per-core usage bars + RAM usage gauge, refreshed every 500ms

Tab 2 — Controller: Running process list + launch / pause / resume / kill controls

Tab 3 — Cleaner: Parallel scan of temp/cache directories, file list, selective delete

All three tabs must be built. Partial credit is available per tab.

A broken Tab 2 does not affect marks for Tab 1 or Tab 3.

Tab 1 — Live Hardware Monitor (30 Marks)

Tab 1 is the foundation of SysForge. It reads real hardware data from Windows and displays it as live updating progress bars and numeric values. The data must come from the operating system — not from a timer that adds random numbers to a fake bar.

1.1 What must be displayed

- One progress bar per logical CPU core, showing that core's current usage percentage (0–100%)
- A single overall CPU usage bar (average of all cores)
- RAM used (in MB and as a percentage of total)
- Total RAM installed (displayed once at startup, does not need to update)
- All values must update every 500 milliseconds
- A timestamp showing the time of the last successful data refresh (HH:MM:SS format)

1.2 Windows API to use

You must use one of the following to read CPU and RAM data. No other method is acceptable.

Option A — PDH (Performance Data Helper) — Recommended for C++

```
#include <pdh.h>
#pragma comment(lib, "pdh.lib")

PDH_HQUERY query;   PDH_HCOUNTER counter;
PdhOpenQuery(NULL, 0, &query);
PdhAddCounter(query, L"\\Processor(0)\\% Processor Time", 0, &counter);
PdhCollectQueryData(query);   Sleep(100);   PdhCollectQueryData(query);
PDH_FMT_COUNTERVALUE val;
PdhGetFormattedCounterValue(counter, PDH_FMT_DOUBLE, NULL, &val);
double cpu0_usage = val.doubleValue;   // 0.0 to 100.0
```

For all cores: use counter path \\Processor(_Total)\\% Processor Time for overall, and \\Processor(N)\\% Processor Time for core N (0-indexed).

Option B — GetSystemTimes() — Simpler for C++

```
FILETIME idle, kernel, user;
GetSystemTimes(&idle, &kernel, &user);
Sleep(500);
// Call again and compute delta
// CPU% = 1.0 - (idle_delta / (kernel_delta + user_delta))
```

For per-core: use GetSystemInfo() to get number of cores, then NtQuerySystemInformation (undocumented but widely used) or PDH per-core counters.

Option C — Python with psutil — Simplest

```
import psutil
cpu_per_core = psutil.cpu_percent(interval=0.5, percpu=True) # list of floats
ram = psutil.virtual_memory()
ram_used_mb = ram.used / 1024**2
ram_percent = ram.percent
```

GUI: use tkinter with a after(500, refresh_fn) loop — never call psutil from the main thread. Use threading.Thread to collect data, queue.Queue to pass it to the GUI thread safely.

1.3 Background thread requirement

The hardware polling must run in a dedicated background thread, not in a GUI timer callback. The background thread wakes every 500ms, reads the data, and posts an update message to the GUI thread. The GUI thread only draws — it never calls hardware APIs directly.

Required threading pattern for Tab 1

C++: CreateThread() or std::thread for the polling thread.

PostMessage(hwnd, WM_USER+1, cpu_val, ram_val) to send data to GUI.

GUI handles WM_USER+1 in WndProc and updates the progress bars.

Python: threading.Thread(target=poll_hardware, daemon=True).start()

poll_hardware puts data into queue.Queue every 500ms.

GUI uses root.after(500, check_queue) to pull from queue and update widgets.

NEVER call Sleep() or time.sleep() on the main/GUI thread.

NEVER update GUI widgets directly from the polling thread.

Both of the above will either freeze your window or crash your app.

1.4 Stress test demonstration (mandatory for full marks)

Your submission must include a built-in stress test button on Tab 1. When clicked, it launches N parallel threads (N = number of logical cores) that each spin in a CPU-bound loop for 10 seconds. The examiner must be able to click this button and watch all CPU core bars visibly rise to near 100% within 2 seconds. If the bars do not respond, it proves the monitoring is not reading real data.

Stress test thread (C++)

```
void stress_worker() {
    auto end = std::chrono::steady_clock::now() + std::chrono::seconds(10);
    volatile double x = 1.0;
    while (std::chrono::steady_clock::now() < end)
        x = x * 1.0000001 + 0.0000001; // prevent compiler optimisation
}

// Launch one thread per core:
int n = std::thread::hardware_concurrency();
for (int i = 0; i < n; i++)
    std::thread(stress_worker).detach();
```

1.5 GPU monitoring (bonus — +10 marks)

GPU monitoring is optional bonus — not required, not penalised if absent

For NVIDIA GPUs: use NVML (NVIDIA Management Library)

```
nvmlInit(); nvmlDeviceGetHandleByIndex(0, &device);
nvmlDeviceGetUtilizationRates(device, &util);
gpu_usage = util.gpu;    gpu_mem = util.memory;
```

For integrated Intel/AMD GPU: use PDH counter:

```
\\GPU Engine(*)\\Utilization Percentage
```

(requires Windows 10 1803 or later)

Display: one GPU usage bar + GPU memory used/total.

If no GPU detected: display 'GPU: Not available' — do not crash.

Tab 1 Rubric

Assessment Criterion	Marks	How Examiner Verifies
Per-core CPU bars displayed and count matches actual core count on machine	6	Open Task Manager > Performance tab. Core count must match. Wrong count = 0.
CPU values accurate: within 10% of Task Manager reading at same moment	7	Examiner reads both simultaneously. More than 10% off on average = partial credit only.
RAM used and total accurate: within 50MB of Task Manager reading	5	Read both simultaneously. Off by more than 50MB = 0 for RAM criterion.
Values update every 500ms — timestamp confirms live refresh	4	Watch timestamp. Must change every 0.5 seconds. Static = 0.
Background polling thread used — GUI never freezes	4	Click stress test during refresh. Window must remain responsive. Any freeze = 0.
Stress test button launches correct number of threads, bars visibly spike	4	Click stress test. All cores must visibly rise. Bars not moving = data is fake.
GPU monitoring working (bonus)	10	GPU bar appears. Value changes when GPU load applied. No GPU = 'Not available' shown.

Assessment Criterion	Marks	How Examiner Verifies
TAB 1 TOTAL	30 (+10)	

Tab 2 — Process Controller (35 Marks)

Tab 2 gives the user control over running Windows processes. It shows a live list of all processes currently running on the machine, and provides buttons to launch a new process, pause it, resume it, and kill it. This is the hardest tab to implement correctly because Windows process control requires specific API calls that must be researched and used precisely.

2.1 Process list requirements

- **Display a scrollable table with one row per running process**
- **Each row must show: Process Name, PID (Process ID), CPU usage (%), RAM usage (MB), Status (Running / Suspended)**
- **The list must refresh every 2 seconds automatically in a background thread**
- **The list must be sortable by clicking column headers (sort by Name, PID, CPU, or RAM)**
- **A search/filter box: typing filters the list to show only processes whose name contains the typed text**

2.2 Windows API for process enumeration

C++ — EnumProcesses (recommended)

```
#include <psapi.h>
#pragma comment(lib, "psapi.lib")

DWORD pids[1024]; DWORD needed;
EnumProcesses(pids, sizeof(pids), &needed);
int count = needed / sizeof(DWORD);

for (int i = 0; i < count; i++) {
    HANDLE h = OpenProcess(PROCESS_QUERY_INFORMATION|PROCESS_VM_READ,
FALSE, pids[i]);
    if (h) {
        TCHAR name[MAX_PATH];
        GetModuleBaseName(h, NULL, name, MAX_PATH); // process name
        // For memory: GetProcessMemoryInfo(h, &pmc, sizeof(pmc))
        CloseHandle(h);
    }
}
```

Python — psutil (recommended for Python students)

```
import psutil
procs = []
for proc in
psutil.process_iter(['pid', 'name', 'cpu_percent', 'memory_info', 'status']):
    try:
        procs.append(proc.info)
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        pass
```

Note: `cpu_percent` on first call returns 0.0 — call it twice with a 0.1s interval.

Always wrap in try/except — processes can die between enumeration and querying.

2.3 Parallel process enumeration

Enumerating all processes and querying CPU + RAM for each is slow if done sequentially — on a busy machine with 200+ processes, a sequential scan takes 300–500ms. You must parallelise the enumeration using OpenMP or a thread pool: divide the list of PIDs among threads, each thread queries its subset of processes, results are merged into the display list.

Why this matters

Without parallelism: 200 processes x 2ms per query = 400ms per refresh — GUI visibly lags.

With 4 threads: 50 processes per thread x 2ms = 100ms — acceptable for a 2-second refresh cycle.

The examiner will test with a loaded machine. Sequential enumeration will miss the 2-second window.

2.4 Process control operations

Four buttons must work on the selected process in the list:

Launch

- A text field where the user types a full executable path (e.g. `C:\Windows\System32\notepad.exe`)
- A Browse button to open a file picker dialog
- A Launch button that starts the process using `CreateProcess()` (C++) or `subprocess.Popen()` (Python)

- **After launch, the new process must appear in the process list within 2 seconds**

Suspend (Pause)

Suspending a process on Windows requires suspending all of its threads. There is no single Windows API call to suspend an entire process — you must enumerate the process threads and suspend each one.

Suspend all threads of a process (C++)

```
HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
THREADENTRY32 te; te.dwSize = sizeof(te);
if (Thread32First(snap, &te)) {
    do {
        if (te.th32OwnerProcessID == target_pid) {
            HANDLE t = OpenThread(THREAD_SUSPEND_RESUME, FALSE,
te.th32ThreadID);
            SuspendThread(t);
            CloseHandle(t);
        }
    } while (Thread32Next(snap, &te));
}
CloseHandle(snap);
```

Python: use ctypes to call NtSuspendProcess or enumerate threads via win32api.

Simpler Python alternative: `import signal; os.kill(pid, signal.SIGSTOP)` (works on WSL only)

Resume

Same as Suspend but call `ResumeThread()` instead of `SuspendThread()` on each thread. The Status column in the process list must change from Suspended to Running after resume.

Kill

```
C++: TerminateProcess(OpenProcess(PROCESS_TERMINATE, FALSE, pid), 1);
Python: psutil.Process(pid).kill() or os.kill(pid, signal.SIGKILL)
```

After kill: the process must disappear from the list within 2 seconds.

IMPORTANT: Do not allow killing system-critical processes (PID < 10, or process name is 'System', 'smss.exe', 'csrss.exe', 'wininit.exe'). Show an error dialog instead.

2.5 Examiner demonstration sequence

The examiner will perform this exact sequence during grading

1. Open SysForge. Navigate to Tab 2. Wait for process list to populate.
2. Verify process count is plausible (should be 50–200 processes on a normal machine).
3. Open Windows Task Manager and compare PIDs for 3 random processes.
4. Type 'notepad' in the filter box. List must filter to show only notepad processes (or none).
5. Click Browse, select C:\Windows\System32\notepad.exe, click Launch.
6. Verify notepad opens AND appears in the SysForge process list within 2 seconds.
7. Select the notepad process in the list. Click Suspend. Verify notepad window is unresponsive.
8. Click Resume. Verify notepad becomes responsive again.
9. Click Kill. Verify notepad closes AND disappears from the SysForge list within 2 seconds.

Tab 2 Rubric

Assessment Criterion	Marks	How Examiner Verifies
Process list populates with real processes matching Windows Task Manager	7	Compare 3 random PIDs with Task Manager. All 3 must match. Fake/hardcoded list = 0 for entire tab.
All 5 columns shown: Name, PID, CPU%, RAM(MB), Status	4	All 5 columns visible. CPU and RAM must be non-zero for active processes.
List refreshes every 2 seconds from background thread — GUI stays responsive	4	New process launched externally appears within 2s. Any GUI freeze during refresh = 0.
Parallel enumeration: OpenMP or thread pool used to query process data	4	Inspect code. Sequential loop over all PIDs = 0. Parallel division = 4.
Column sort working (click header sorts by that column)	3	Click each header. List must reorder. Clicking again must reverse order.
Filter box: typing filters list to matching process names in real time	3	Type 'chrome'. Only chrome processes shown. Clear field: all processes return.
Launch: CreateProcess/Popen launches app, appears in list within 2 seconds	4	Examiner sequence step 5-6. Both GUI app opens AND list updates.

Assessment Criterion	Marks	How Examiner Verifies
Suspend: all threads of process suspended, window becomes unresponsive	4	Examiner sequence step 7. Notepad must visibly freeze (cannot type or move it).
Resume: process resumes, status changes to Running	2	Examiner sequence step 8. Notepad must become interactive again.
Kill: process terminates, disappears from list within 2 seconds	2	Examiner sequence step 9. Notepad closes. List updates.
TAB 2 TOTAL	37	Rubric intentionally has 37 sub-marks — examiner scales to 35.

Tab 3 — Parallel System Cleaner (25 Marks)

Tab 3 scans the Windows file system for temporary, cached, and redundant files — the kind of files a disk cleaner like CCleaner removes. The scan must be parallelised using OpenMP or a thread pool. The user sees a live-updating list of found files and chooses which ones to delete.

3.1 Directories to scan

Your cleaner must scan all of the following directories. Some may not exist on every machine — handle missing directories gracefully (skip them, do not crash):

Required scan targets

C:\Windows\Temp	(Windows temp files)
%TEMP%	(resolves to user temp, e.g. C:\Users\<name>\AppData\Local\Temp)
C:\Windows\Prefetch	(Windows prefetch data)
C:\Windows\SoftwareDistribution\Download	(Windows Update cache)
%LOCALAPPDATA%\Google\Chrome\User Data\Default\Cache	(Chrome cache)
%LOCALAPPDATA%\Microsoft\Edge\User Data\Default\Cache	(Edge cache)
C:\\$Recycle.Bin	(Recycle Bin contents)

Not all of these will exist on every machine. Scan what exists. Skip what does not.
Do not attempt to scan C:\ recursively — that would take hours.

3.2 Scan requirements

- Each directory must be scanned in a separate thread (one thread per directory = 7 threads)
- Within each directory, file enumeration must also be parallelised if the directory contains more than 100 files
- During scanning, show a live progress indicator: 'Scanning... X files found so far'
- The progress indicator must update every 250ms — not only when the scan completes
- Show which directory is currently being scanned in a status label
- Total scan time must be displayed after completion in seconds to 2 decimal places

3.3 File list display

After scanning completes, display a table with one row per found file:

- **Filename (full path)**
- **File size in KB or MB (auto-select appropriate unit)**
- **Last modified date (DD/MM/YYYY format)**
- **Category: Temp / Prefetch / Update Cache / Browser Cache / Recycle Bin**
- **A checkbox per row — checked by default, unchecked to exclude from deletion**

Above the table, show summary statistics:

- Total files found across all directories
- Total size of all found files in MB
- Breakdown by category: e.g. Temp: 234 files (45 MB) | Prefetch: 128 files (12 MB) | ...

3.4 Delete operation

A Delete Selected button at the bottom of the tab. When clicked:

1. **Show a confirmation dialog: 'You are about to delete X files totalling Y MB. This cannot be undone. Proceed?'**
2. **If confirmed: delete all checked files in parallel (one thread per category, or OpenMP parallel for over the checked file list)**
3. **Show a progress bar during deletion: 'Deleting... X of Y files'**
4. **After deletion: update the file list to remove deleted entries and recalculate summary statistics**
5. **Show a completion message: 'Deleted X files. Freed Y MB.'**

Safety rules — mandatory

NEVER delete files outside the scan targets listed in Section 3.1.

NEVER delete files from C:\Windows\System32 or C:\Program Files.

NEVER delete files without the confirmation dialog.

If a file cannot be deleted (in use, permissions): skip it and report it in a 'Failed to delete' list.

Do not delete the scan target directories themselves — only their contents.

3.5 Parallel scan architecture

Recommended parallel scan pattern (C++)

```
std::vector<std::string> targets = { temp_dir, user_temp, prefetch, ... };
std::vector<FileInfo> all_results;
std::mutex results_mutex;

#pragma omp parallel for num_threads(targets.size())
for (int i = 0; i < targets.size(); i++) {
    auto files = scan_directory(targets[i]);    // recursive file walk
    std::lock_guard<std::mutex> lock(results_mutex);
    all_results.insert(all_results.end(), files.begin(), files.end());
}
```

Python: use `concurrent.futures.ThreadPoolExecutor(max_workers=7)` with one submit per directory.

Tab 3 Rubric

Assessment Criterion	Marks	How Examiner Verifies
Scan starts when button clicked, live progress updates every 250ms	4	Watch counter during scan. Must visibly increment. Static until done = 0.
All existing scan targets scanned, missing ones skipped without crash	4	Manually check which targets exist on exam machine. All present ones must appear in results.
Each directory scanned in a separate parallel thread	5	Inspect code. Sequential loop over directories = 0. Parallel (OpenMP or thread pool) = 5.
File list table shows all 5 columns with correct data	4	Open one result file in Explorer. Verify size and date match what SysForge shows.
Summary statistics correct: total count, total MB, per-category breakdown	3	Count files in one category manually (e.g. Prefetch). Must match SysForge count.
Confirmation dialog appears before any deletion	2	Click Delete. Dialog must appear. No dialog = 0 immediately.
Delete runs in parallel, progress bar shown, completion message accurate	3	Delete a small batch. Progress bar must appear. Freed MB in message must match file sizes.
TAB 3 TOTAL	25	

Live Proof Panel — The Anti-Fabrication Mechanism (10 Marks)

Toggled with the F2 key, the Live Proof Panel is a small overlay or side panel that shows technical evidence that your background threads are genuinely running. It serves the same purpose as a drug test: the result must match what the body is actually doing.

The panel must show the following, updated every second:

```
[ SYSFORGE LIVE PROOF PANEL ]   (F2 to toggle)

BACKGROUND THREADS
  Monitor thread:    ACTIVE    last wake: 14:23:07.441    interval: 501ms
  Process thread:    ACTIVE    last wake: 14:23:06.112    interval: 2003ms
  Cleaner thread:    IDLE      last wake: 14:22:51.003    (scan completed)

MONITOR DATA SOURCE
  API used:          PDH
  Query handle:      0x000001A4  (non-zero = open)
  Last raw CPU val:  [12.4, 87.3, 45.1, 22.8, ...]  (one per core)
  Last raw RAM val:  used=6842MB  total=16384MB

PROCESS THREAD
  Enumeration threads: 4
  PIDs scanned this refresh: 187
  Scan duration: 94ms
  Processes in list: 183  (4 filtered by access denied)

CLEANER (last scan)
  Threads used: 7  (one per directory)
  Total scan time: 3.42s
  Files found: 1,847
```

Why this panel cannot be faked

'last wake' timestamp changes every 500ms for the monitor thread. Static = fake.
'Last raw CPU val' must match the bar values visible on Tab 1 at the same moment.
'PIDs scanned this refresh' must be plausible (150-300 on a normal Windows machine).
'Query handle' must be non-zero (a real PDH handle is a non-null pointer value).

The examiner will compare panel values against Tab 1 bars and Task Manager simultaneously.

Any inconsistency is immediate evidence of fabricated or copied data.

Assessment Criterion	Marks	How Examiner Verifies
Panel toggles on/off with F2 without crashing any tab	2	Press F2. Panel appears. Press again. Disappears. All three tabs still live.
Monitor thread timestamps update every 500ms (visibly changing)	3	Watch 'last wake' for 3 seconds. Must change 6 times. Static = 0.
Raw CPU values match Tab 1 bar values at the same moment	3	Read raw values from panel. Compare to bars. Must be consistent within 5%.
Process scan duration and PID count plausible and consistent with actual machine	2	160-300 PIDs expected. Scan duration 50-200ms expected. Outside range = suspicious.
PROOF PANEL TOTAL	10	

Grading Summary

Tab 1 — Monitor	Tab 2 — Controller	Tab 3 — Cleaner	Proof Panel
30 marks	35 marks	25 marks	10 marks

Total: 100 marks | GPU monitoring bonus: +10 marks | Tab 2 rubric scales from 37 to 35.

Grade Boundaries

Marks	Grade	Typical Submission Profile
88–100	A+	All three tabs live and accurate. Live proof panel shows real thread IDs and timestamps. GPU bonus implemented. Stress test visibly spikes CPU bars. Cleaner scans in parallel with thread count shown. Process launch/kill/pause all working. Verbal Q&A: answers every question instantly.
74–87	A	All three tabs working. Minor inaccuracies in one tab (e.g., RAM off by small margin). Live proof panel present. Two of three process controls working (e.g., launch and kill but not pause).
58–73	B	Two tabs working correctly. Third tab opens but data is wrong or static. Background threads present in code but one has a race condition. Stress test runs but bars do not visibly respond.
42–57	C	One tab fully working (usually Monitor). Second tab partially done. Process list shows but cannot launch/kill. Cleaner opens but scan is sequential with no thread count shown.
25–41	D	App window opens. One tab partially works. No genuine background threads — GUI freezes during any operation.
0–24	F	Does not compile or run. Submitted code unrelated to task. Single static window with no live data.

Examiner Verbal Assessment Question Bank

Pick 3 questions per student immediately after submission. A student who wrote their own code answers within 20 seconds. A student who used Claude Code or copied code will not know where in their own files these answers are.

Tab 1 questions

- Show me the line in your code where PdhCollectQueryData is called. Why is it called twice with a sleep in between?
- Your CPU bar for core 3 shows 87%. Open Task Manager right now. What does core 3 show there? Why might they differ slightly?
- What Windows message does your polling thread send to the GUI thread when new data is ready? Show me the handler in WndProc.

Tab 2 questions

- Your process list shows 183 processes. Task Manager shows 187. What are the 4 processes your app missed, and why?
- I selected notepad and clicked Suspend. Your code called SuspendThread. How many threads did notepad have, and how did you find out?
- Show me the exact OpenProcess flags you used for enumeration. Why did you choose those specific flags and not PROCESS_ALL_ACCESS?

Tab 3 questions

- Your cleaner found 1,847 files in 3.42 seconds using 7 threads. If you used 1 thread, how long would you estimate it would take? Why?
- You scanned %TEMP% and found 234 files. Show me how to navigate to that folder in Explorer right now and count them manually.
- What happens in your code if the Chrome cache folder does not exist on this machine? Show me that code path.

Examiner Step-by-Step Grading Checklist

Step 1 — Compilation check

Compile the submission using standard flags. Note any errors. Do not fix student code.

```
C++: cl /openmp sysforge.cpp /link pdh.lib psapi.lib OR g++ -fopenmp  
sysforge.cpp -lpdh  
Python: python sysforge.py (note any missing imports)
```

Step 2 — Tab 1 verification (5 minutes)

Open Task Manager > Performance tab side by side with SysForge.

Note core count shown in SysForge vs actual core count.

Click Stress Test. Watch both monitors. SysForge bars must spike within 2 seconds.

Wait 10 seconds. Verify bars return to idle levels after stress test ends.

Press F2. Read raw CPU values from Proof Panel. Compare to Tab 1 bars.

Step 3 — Tab 2 verification (5 minutes)

Compare 3 random PIDs between SysForge and Task Manager.

Type 'notepad' in filter. Verify filter works.

Launch C:\Windows\System32\notepad.exe from SysForge. Verify it opens.

Verify notepad appears in SysForge list within 2 seconds.

Suspend notepad. Try to type in it — must be unresponsive.

Resume notepad. Try to type in it — must be responsive.

Kill notepad. Verify it closes and disappears from list.

Step 4 — Tab 3 verification (3 minutes)

Click Scan. Watch live progress counter update during scan.

After completion: verify at least one scan target found files.

Open Windows Explorer and manually navigate to %TEMP%.

Compare file count in Explorer to SysForge Temp category count.

Select 5 small files in SysForge. Click Delete. Confirm dialog appears. Confirm.

Verify those 5 files no longer exist in Explorer after deletion.

Step 5 — Verbal assessment (3 questions, 3 minutes)

Pick 3 questions from the verbal question bank above.

Student who wrote their own code: answers in under 20 seconds with code reference.

Student who used AI: cannot locate the relevant code, gives vague generic answer.

Record pass/fail per question for your own notes — use in borderline grade decisions.

Appendix — Windows API Quick Reference

Recommended GUI frameworks

C++ options (all free, all work on Windows):

Win32 API (raw): Maximum control, complex. Use for high marks on API knowledge.

Qt 6 (free): `sudo choco install qt6` OR download from qt.io. Best for clean GUI.

wxWidgets: Good balance. `wxFrame`, `wxPanel`, `wxProgressBar` built-in.

ImGui: Immediate-mode GUI, excellent for debug panels.
github.com/ocornut/imgui

Python options:

tkinter: Built into Python. Use `ttk.Progressbar`, `ttk.Treeview` for process list.

PyQt6: `pip install PyQt6`. Most powerful Python GUI for this task.

wxPython: `pip install wxpython`. Good alternative to PyQt.

Key Windows API calls summary

Hardware monitoring:

PDH: `PdhOpenQuery`, `PdhAddCounter`, `PdhCollectQueryData`, `PdhGetFormattedCounterValue`

RAM: `GlobalMemoryStatusEx()` -> `MEMORYSTATUSEX` struct

Core count: `GetSystemInfo()` -> `SYSTEM_INFO.dwNumberOfProcessors`

Process management:

List processes: `EnumProcesses()` + `OpenProcess()` + `GetModuleBaseName()`

Launch process: `CreateProcess(NULL, cmdline, ...)`

Kill process: `TerminateProcess(handle, exit_code)`

Suspend/Resume: `CreateToolhelp32Snapshot` + `Thread32First/Next` + `SuspendThread/ResumeThread`

Process memory: `GetProcessMemoryInfo()` -> `PROCESS_MEMORY_COUNTERS`

File system:

Enumerate files: `FindFirstFile()` + `FindNextFile()` (C++) or `os.walk()` (Python)

File size: `GetFileAttributesEx()` -> `WIN32_FILE_ATTRIBUTE_DATA.nFileSizeLow`

Delete file: `DeleteFile()` (C++) or `os.remove()` (Python)

Expand %TEMP%: `ExpandEnvironmentStrings(L"%TEMP%", buf, MAX_PATH)`

```
Python equiv:      os.environ.get('TEMP')   or  
os.path.expandvars('%TEMP%')
```

END OF EXAM PAPER — SYSFORGE

There are no images. There are no enemies. There are no text files to index.

There is only your machine, running right now, and your job is to read it.

Open Task Manager. That is what your app must look like from the inside.